

INTRO TO ASSEMBLY LANGUAGE CHEAT SHEET

Registers

Description	64-bit Register (8-bytes)	8-bit Register (1-bytes)
Data/Arguments Registers		
Syscall Number/Return value	rax	al
Callee Saved	rbx	bl
1st arg	rdi	dil
2nd arg	rsi	sil
3rd arg	rdx	d1
4th arg - Loop Counter	rcx	c1
5th arg	r8	r8b
6th arg	r9	r9b
Pointer Registers		
Base Stack Pointer	rbp	bp1
Current/Top Stack Pointer	rsp	sp1
Instruction Pointer 'call only'	rip	ip1

Assembly and Disassembly

Command	Description
<code>nasm -f elf64 helloWorld.s</code>	Assemble code
<code>ld -o helloWorld helloWorld.o</code>	Link code
<code>ld -o fib fib.o -lc --dynamic-linker /lib64/ld-linux-x86-64.so.2</code>	Link code with libc functions
<code>objdump -M intel -d helloWorld</code>	Disassemble <code>.text</code> section
<code>objdump -M intel --no-show-raw-insn --no-addresses -d helloWorld</code>	Show binary assembly code
<code>objdump -sj .data helloWorld</code>	Disassemble <code>.data</code> section

GDB

Command	Description
<code>gdb -q ./helloWorld</code>	Open binary in gdb
<code>info functions</code>	View binary functions
<code>info variables</code>	View binary variables
<code>registers</code>	View registers
<code>disas _start</code>	Disassemble label/function
<code>b _start</code>	Break label/function
<code>b *0x401000</code>	Break address
<code>r</code>	Run the binary

Command	Description
<code>x/4xg \$rip</code>	Examine register "x/ count-format-size \$register"
<code>si</code>	Step to the next instruction
<code>s</code>	Step to the next line of code
<code>ni</code>	Step to the next function
<code>c</code>	Continue to the next break point
<code>patch string 0x402000 "Patched!\\x0a"</code>	Patch address value
<code>set \$rdx=0x9</code>	Set register value

Assembly Instructions

Instruction	Description	Example
Data Movement		
<code>mov</code>	Move data or load immediate data	<code>mov rax, 1 -> rax = 1</code>
<code>lea</code>	Load an address pointing to the value	<code>lea rax, [rsp+5] -> rax = rsp+5</code>
<code>xchg</code>	Swap data between two registers or addresses	<code>xchg rax, rbx -> rax = rbx, rbx = rax</code>
Unary Arithmetic Instructions		
<code>inc</code>	Increment by 1	<code>inc rax -> rax++ or rax += 1 -> rax = 2</code>
<code>dec</code>	Decrement by 1	<code>dec rax -> rax-- or rax -= 1 -> rax = 0</code>

Instruction	Description	Example
Binary Arithmetic Instructions		
add	Add both operands	add rax, rbx -> rax = 1 + 1 -> 2
sub	Subtract Source from Destination (<i>i.e</i> rax = rax - rbx)	sub rax, rbx -> rax = 1 - 1 -> 0
imul	Multiply both operands	imul rax, rbx -> rax = 1 * 1 -> 1
Bitwise Arithmetic Instructions		
not	Bitwise NOT (<i>invert all bits, 0->1 and 1->0</i>)	not rax -> NOT 00000001 -> 11111110
and	Bitwise AND (<i>if both bits are 1 -> 1, if bits are different -> 0</i>)	and rax, rbx -> 00000001 AND 00000010 -> 00000000
or	Bitwise OR (<i>if either bit is 1 -> 1, if both are 0 -> 0</i>)	or rax, rbx -> 00000001 OR 00000010 -> 00000011
xor	Bitwise XOR (<i>if bits are the same -> 0, if bits are different -> 1</i>)	xor rax, rbx -> 00000001 XOR 00000010 -> 00000011
Loops		
mov rcx, x	Sets loop (rcx) counter to x	mov rcx, 3
loop	Jumps back to the start of loop until counter reaches 0	loop exampleLoop

Branching

Instruction	Description	Example
<code>jmp</code>	Jumps to specified label, address, or location	<code>jmp loop</code>
<code>jz</code>	Destination equal to Zero	<code>D = 0</code>
<code>jnz</code>	Destination Not equal to Zero	<code>D != 0</code>
<code>js</code>	Destination is Negative	<code>D < 0</code>
<code>jns</code>	Destination is Not Negative (i.e. 0 or positive)	<code>D >= 0</code>
<code>jg</code>	Destination Greater than Source	<code>D > S</code>
<code>jge</code>	Destination Greater than or Equal Source	<code>D >= S</code>
<code>j1</code>	Destination Less than Source	<code>D < S</code>
<code>jle</code>	Destination Less than or Equal Source	<code>D <= S</code>
<code>cmp</code>	Sets RFLAGS by subtracting second operand from first operand (i.e. <i>first - second</i>)	<code>cmp rax, rbx -> rax - rbx</code>
Stack		
<code>push</code>	Copies the specified register/address to the top of the stack	<code>push rax</code>
<code>pop</code>	Moves the item at the top of the stack to the specified register/address	<code>pop rax</code>
Functions		
<code>call</code>	push the next instruction pointer rip to the stack, then jumps to the specified procedure	<code>call printMessage</code>
<code>ret</code>	pop the address at rsp into rip , then jump to it	<code>ret</code>

Functions

Command	Description
<code>cat /usr/include/x86_64-linux-gnu/asm/unistd_64.h grep write</code>	Locate write syscall number
<code>man -s 2 write</code>	write syscall man page
<code>man -s 3 printf</code>	printf libc man page

Syscall Calling Convention

- 1. Save registers to stack
- 2. Set its syscall number in **rax**
- 3. Set its arguments in the registers
- 4. Use the **syscall** assembly instruction to call it

Function Calling Convention

- 1. **Save Registers** on the stack (**Caller Saved**)
- 2. Pass **Function Arguments** (*like syscalls*)
- 3. Fix **Stack Alignment**
- 4. Get Function's **Return Value** (*in rax*)

Shellcoding

Command	Description
<code>pwn asm 'push rax' -c 'amd64'</code>	Instruction to shellcode
<code>pwn disasm '50' -c 'amd64'</code>	Shellcode to instructions
<code>python3 shellcoder.py helloworld</code>	Extract binary shellcode
<code>python3 loader.py '4831..0f05</code>	Run shellcode
<code>python assembler.py '4831..0f05</code>	Assemble shellcode into binary

Shellcraft

Command	Description
<code>pwn shellcraft -l 'amd64.linux'</code>	List available syscalls
<code>pwn shellcraft amd64.linux.sh</code>	Generate syscalls shellcode
<code>pwn shellcraft amd64.linux.sh -r</code>	Run syscalls shellcode

Msfvenom

<code>msfvenom -l payloads grep 'linux/x64'</code>	List available syscalls
<code>msfvenom -p 'linux/x64/exec' CMD='sh' -a 'x64' --platform 'linux' -f 'hex'</code>	Generate syscalls shellcode
<code>msfvenom -p 'linux/x64/exec' CMD='sh' -a 'x64' --platform 'linux' -f 'hex' -e 'x64/xor'</code>	Generate encoded syscalls shellcode

Shellcoding Requirements

- 1. Does not contain variables
- 2. Does not refer to direct memory addresses
- 3. Does not contain any NULL bytes `00`